

基于改进蚁群优化算法的船舶管路布局设计

董宗然^{1,2}, 陈恒¹, 卞璇屹³, 楼偶俊¹

1. 大连外国语大学 软件学院, 辽宁 大连 116044

2. 大连外国语大学 图书情报研究所, 辽宁 大连 116044

3. 大连理工大学 船舶工程学院, 辽宁 大连 116024

摘要:为了实现在多种约束和目标限制下,能以较短时间为工程师提供多种优质船舶布管方案,提出一种基于改进蚁群优化算法的新型船舶管路布局设计方法。该方法基于网格空间分解模型,将适应度函数中的优化目标进行规范化,对蚁群优化算法的关键步骤,如:蚂蚁行进方向选择、信息素更新机制、蚂蚁寻路过程等进行改进,采用最优解集合保存适应度相同但管路布局效果不同的多个优解,引入辅助点策略和并行计算机制提升算法整体寻优能力和效率。通过仿真算例验证了算法的有效性和先进性。

关键词:船舶管路布局设计;路径设计;蚁群优化;并行计算

文献标志码:A **中图分类号:**TP399 **doi:**10.3778/j.issn.1002-8331.2212-0298

Ship Pipe Layout Design Based on Improved Ant Colony Optimization

DONG Zongran^{1,2}, CHEN Heng¹, BIAN Xuanyi³, LOU Oujun¹

1. School of Software, Dalian University of Foreign Languages, Dalian, Liaoning 116044, China

2. Institute of Library and Information, Dalian University of Foreign Languages, Dalian, Liaoning 116044, China

3. School of Naval Architecture and Ocean Engineering, Dalian University of Technology, Dalian, Liaoning 116024, China

Abstract: In order to provide engineers with a variety of high-quality ship pipe layouts in a relatively short time under multiple constraints and objectives, a novel ship pipe layout design method based on the modified ant colony optimization (MdACO) is proposed. The method works in a grid-decomposition space model, firstly the optimization objectives of the fitness function are normalized, and then the key steps of MdACO are improved, such as the selection of ant-moving direction, the mechanism of pheromone update, and the procedure of ant route searching. The best solution set is used to save lots of optimization individuals with the same fitness but different layout effects. Moreover, the auxiliary connection point and parallel computing are integrated to improve the searching ability and efficiency of the algorithm. Finally, the effectiveness and advancement of the algorithm are verified by the simulation test.

Key words: ship pipe layout design; route design; ant colony optimization; parallel computing

船舶管路用于在船舶设备间传送油、气、水、电等资源,对船舶功能和性能表现有重要影响。船舶管路设计可分为:初步设计、功能设计、详细设计、生产设计等阶段^[1],其中管路路径布局设计属于详细设计阶段的主要工作,并为后续设计活动提供基础。船舶内部空间常按功能分为不同区域,如:机舱区、生活区、甲板区、特殊功能区,不同区域的管路设计目标不同,机舱区域设备多、管路数量大、约束复杂,是船舶管路设计的重点和难点^[2]。

船舶管路布局设计本质为在三维空间寻找设备接

口间的连通路径,并满足路径合法性、有效性和优化性。当前主流CAD软件,如CATIA、FORAN、TRIBON、SolidWorks,虽然提供自动步路模块,但只能按照有限模式生成简单管子路径且不考虑避障,需要依赖设计师的人工调整。此外,船舶生产具有订制特点且管路设计常随着设备布局调整返工,导致人工设计工作量大、效率低,因此探索自动管路布局设计算法对提升船舶设计效率具有重要意义,同时也可为航空发动机^[3]、建筑空间^[4]、机电装备^[5]领域的管路或线缆布局,以及机器人路

基金项目:2022年度辽宁省高等学校基本科研项目(LJKMZ20221547)。

作者简介:董宗然(1981—),男,博士,副教授,CCF会员,研究领域为进化计算和船舶管路布局优化,E-mail:dongzongran@163.com;

陈恒(1982—),男,博士,副教授,CCF会员,研究领域为智能信息处理;卞璇屹(1993—),男,博士研究生,研究领域为

船舶管路自动设计与船舶辅助软件系统开发;楼偶俊(1976—),男,博士,副教授,研究领域为人工智能。

收稿日期:2022-12-22 **修回日期:**2023-04-26 **文章编号:**1002-8331(2024)07-0344-11

径规划^[6]等提供借鉴和参考。管路布局相关研究已受到国内外学者广泛关注,代表性研究包括:Park等^[2]最早提出使用单元生成法自动布局船舶机舱内管路;Asmara等^[7]对比了多种船舶管路设计算法的区别并提出将启发式算法(PSO)与确定性算法(A*)结合的求解策略;Fan等^[8-9]最早将蚁群算法应用于船舶管路布局设计,Jiang等^[10]和Wang等^[11]分别采用协同进化和人机结合策略改进蚁群算法求解船舶布管,但算法效率和布局多样性仍有待提高;Liu等^[12]提出了一种基于自适应动态调整策略的蚁群寻路算法,并应用于二维平面管路设计;Niu等^[13-14]提出基于多目标优化算法NSGA-II布局船舶管路,体现出同时提供多种布管方案的价值;笔者前期代表性工作分别对表现出色的多目标进化算法NSGA-II^[15]、MOACO^[16]以及混合启发式算法GA+A*^[17]进行了研究。以上研究方法从求解小规模简单问题发展到求解复杂实际问题,但仍有不足之处,主要原因是船舶管路布局是在复杂约束下的多目标优化问题,求解难度大,单一方法具有局限性,根据用户需求选择适合算法更为可行。蚁群算法模拟蚂蚁觅食的寻路行为,具有并行性的本质特征^[18],但也存在收敛速度慢、易于陷入局部最优等问题。本文提出一种基于并行计算思路的改进蚁群优化布管算法(modified ant colony optimization, MdACO),发挥蚁群搜索的并行属性,以期高效地实现船舶管路布局设计。

后续先介绍船舶管路布局设计问题模型,接着给出改进蚁群布管算法关键步骤和算法框架,再通过算例验证算法可行性和先进性,最后总结全篇,提出展望。

1 船舶管路布局问题模型

1.1 布局空间与管路

布局空间是容纳船舶结构、设备和管路的区域,为了便于算法定量分析,先对布局空间进行网格划分,过程如下:用管路中最小的管径尺寸加上间隔距离作为网格单元边长(L),将三维布局空间划分成立方体网格集合,将障碍物覆盖网格标记为“禁用”状态,将允许管路经过空间网格标记为“自由”状态,对于“自由”网格,进一步通过网格能量值区分网格所在区域对管路的友好程度,将期望管路经过空间网格设为低能量,不期望管路经过空间网格设为高能量,其余自由空间网格设为常规能量,布局空间及其网格状态的示例如图1。

网格位置用(行、列、层)构成的坐标表示,设布局空间左下后角点网格坐标为(0,0,0),则空间中一个网格位置记为 $cell(R_n, C_n, L_n)$ 。确定管路起点 s 和终点 t 所在网格后,管路路径可用连接 s, t 的网格序列表示(如式(1)):

$$\begin{aligned} Path(s \rightarrow t) &: cell(R_s, C_s, L_s) \rightarrow \dots \rightarrow \\ &cell(R_m, C_m, L_m) \rightarrow cell(R_n, C_n, L_n) \rightarrow \dots \rightarrow cell(R_t, C_t, L_t), \\ &|R_m - R_n| + |C_m - C_n| + |L_m - L_n| = 1 \end{aligned} \quad (1)$$

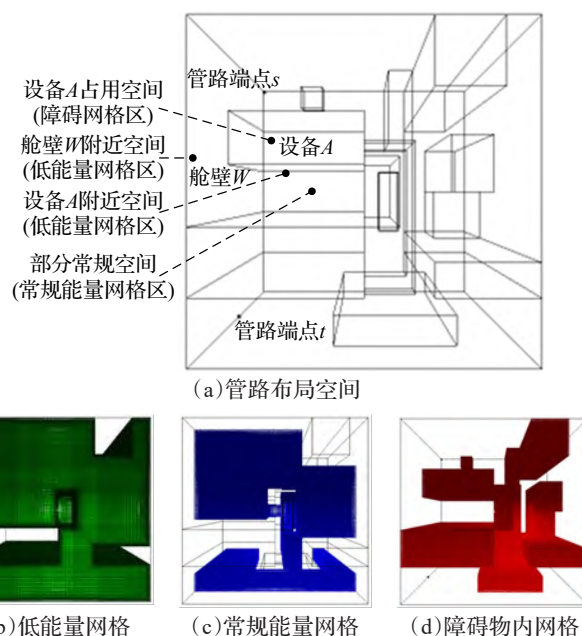


图1 网格分解空间

Fig.1 Cell decomposition space

当布局管路 P 的管径 D_p 大于网格边长 L 时,为使管路不与障碍物干涉,可采用障碍物膨胀法做预处理,在布置管路 P 前先对空间中的障碍物做膨胀处理,再运行寻路算法即可保证管路 P 扩展到原始尺寸也不会与障碍干涉,最后将膨胀空间恢复成初始状态。基于障碍膨胀法布管,具有不同管径的管路可在同一网格空间完成布局^[13,17]。

1.2 布局目标和约束

实际船舶管路布局中约束和目标众多,算法不能全部考虑,因此本文仅处理便于量化和算法求解的目标和约束,实际工作中管路设计师还需结合人工经验对结果进一步选择。

约束条件:

- (1) 管路与障碍物(舱室结构、设备、已布局管路、预留空间等)不发生碰撞;
- (2) 管路路径与船体结构正交布置,虽然船舶中存在非正交布置的管路以节省空间和管材,但可由设计师后期在正交结果上调整优化。

优化目标:

- (1) 管路路径长度尽可能短;
- (2) 管路路径折弯尽可能少;
- (3) 管路尽可能靠近支撑面(舱壁、地板、特定设备)布局,方便安装;
- (4) 管路路径尽可能避免“凹兜”结构(竖直方向的U型路径),减少介质阻滞;
- (5) 管路相邻折弯间距不小于限定长度,以满足加工约束或减少加工成本。

1.3 问题形式化

船舶管路布局本质为在一定约束条件下的多目标优化问题,可将1.2节的目标和约束描述为如下形式。

$$\begin{aligned} Objective(path) = & \{\min f_{\text{length}}(path), \min f_{\text{bend}}(path), \\ & \min f_{\text{power}}(path), \min f_{\text{pocket}}(path), \min f_{\text{times}}(path)\} \\ \text{s.t. } g(path) = & 0, h(path) = 0 \end{aligned} \quad (2)$$

其中, $f_{\text{length}}(path)$ 计算路径长度, $f_{\text{bend}}(path)$ 计算路径折弯数目, $f_{\text{power}}(path)$ 计算路径经过位置能量值, $f_{\text{pocket}}(path)$ 计算路径在竖直方向形成“凹兜”结构数目, $f_{\text{times}}(path)$ 计算路径折弯间距小于限制长度次数。 $g(path)=0$ 表示路径不与布局空间发生碰撞, $h(path)=0$ 表示路径与地板或墙面正交布置(折弯为直角)。

多目标优化问题的求解有多种方法^[19], 船舶管路布局设计领域主要用加权求和法^[12, 17]和多目标优化算法^[13-16]。加权求和法为不同目标分配权重, 然后对子目标加权求和, 将多目标优化转化为单目标优化; 多目标优化算法则基于非支配排序给出互不占优个体形成的 Pareto 解集。一般而言, 加权求和法一次计算给出单一优解, 但可根据不同权重设置在多次计算中得到满足用户不同偏好的优解; 多目标算法通过一次计算便可生成具有不同布局特征的 Pareto 解集, 但求解过程中缺少人工经验, 需要用户在 Pareto 解集中进一步选择满足偏好的解。单目标算法通过为各目标指定权重, 直接反映设计师偏好, 解的生成过程相比多目标算法更受控, 因此本文采用基于加权求和的蚁群算法布置管路, 并将具有相同适应值但布局不同的优解进行保存, 与多目标算法一样为设计师提供多种布局参考。

为了规避目标间的量纲差异, 定义规范化的适应度函数如式(3):

$$\begin{aligned} fitness(path) = & w_{\text{length}} \left(\frac{length_{\max} - f_{\text{length}}(path)}{length_{\max} - length_{\min}} \right) + \\ & w_{\text{bend}} \left(\frac{bend_{\max} - f_{\text{bend}}(path)}{bend_{\max} - bend_{\min}} \right) + \\ & w_{\text{power}} \left(\frac{power_{\max} - f_{\text{power}}(path)}{power_{\max} - power_{\min}} \right) + \\ & w_{\text{pocket}} \left(\frac{pocket_{\max} - f_{\text{pocket}}(path)}{pocket_{\max} - pocket_{\min}} \right) + \\ & w_{\text{times}} \left(\frac{times_{\max} - f_{\text{times}}(path)}{times_{\max} - times_{\min}} \right) \end{aligned} \quad (3)$$

路径适应值 $fitness(path)$ 越大, 所对应路径越好, 其中 w_{length} 、 w_{bend} 、 w_{power} 、 w_{pocket} 、 w_{times} 为各子目标权重系数, 各子目标值越小越好, 前三个目标要尽可能优化, 权重系数 w_{length} 、 w_{bend} 、 w_{power} 的取值建议加和为1, 用户可根据这三个子目标的相对重要程度调整其相对大小关系, 后两个目标取值尽可能为0, w_{pocket} 、 w_{times} 建议取较大值(如100、500等), 以起到惩罚作用。 $length_{\max}$ 、 $length_{\min}$ 、 $\dots$ 、 $times_{\max}$ 、 $times_{\min}$ 等对应搜索到当前代数时, 已搜索个体中长度、折弯数、能量值、凹兜数、违反折弯长度数的最大值和最小值, 称为“界限值”。这些“界

限值”随着搜索过程更新, 因此在将新个体与已保留优解个体比较前, 需要重新计算优解个体的适应值。

2 改进蚁群管路布局设计算法

本章首先介绍改进蚁群管路布局算法(MdACO)的关键设计和改进策略, 然后给出算法整体流程和蚂蚁寻路流程。

2.1 算法关键技术

2.1.1 蚂蚁寻路策略

蚂蚁寻路中最重要的决策是选择每步行进方向, 方向选择决定了路径优劣。蚂蚁从起点到终点的寻路中需要参考启发式信息和信息素浓度确定行进方向。

(1) 启发式信息

研究文献[8-11]中将蚂蚁当前位置 s 与目标位置 t 间的距离倒数作为启发式信息, 启发式信息大的下一网格位置更能吸引蚂蚁, 若仅用距离信息 $distance$ 引导, 则在一些布局环境下(例如 s 和 t 之间有大障碍), 以往ACO算法的蚂蚁将无法逾越障碍到达终点(图2(a)), 因此将路径折弯数和能量值加入启发信息, 启发式信息定义如式(4), 其中距离信息是必须的, 折弯信息和能量信息可由用户单独配置, 用户根据需求决定是否加入, 加入 $bends$ 和 $power$ 后, 合成的启发信息绕障能力更强(图2(b)), 而且生成路径能更好满足折弯少和能量低的优化目标。

$$Heuristic(c) = 1 / (distance(c_{i+1}, c_{\text{end}}) + bends(c_{i-1}, c_i, c_{i+1}) + power(c_{i+1})) \quad (4)$$

c_i 为蚂蚁当前网格位置, c_{i-1} 为前一个网格位置, c_{i+1} 为下一个要探索的网格位置, c_{end} 为终点网格位置。 $distance()$ 计算下一网格与终点网格的欧式距离, $bends()$ 计算下一网格是否会形成以 c_i 为拐点的路径折弯, 如形成折弯则取值为1, 否则取0, $power()$ 计算下一网格引入的能量代价。显然引入代价之和越小的 c_{i+1} 对蚂蚁越有吸引力, 其启发式信息越大。实验发现, 式中距离函数 $distance()$ 、折弯函数 $bends()$ 、能量函数 $power()$ 的取值范围数量级相差不大, 因此采用默认调节系数1即可获得较好效果, 若为了强化某一代价函数在启发式信息中的作用, 可通过增大其调节系数实现。

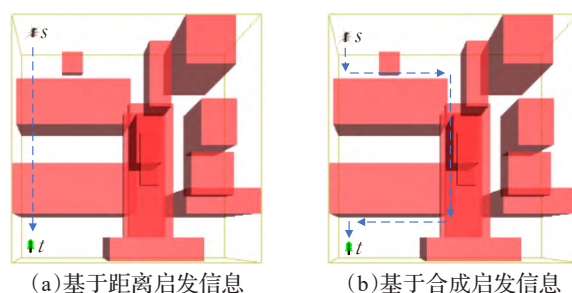


图2 启发式信息的方向引导

Fig.2 Direction guidance of heuristic information

(2) 信息素信息

信息素用于在蚂蚁个体间传递信息,蚂蚁会将信息素释放在经过路径上,信息素伴随时间挥发,蚂蚁倾向于沿信息素浓度高的路径行进。因此短距离路径上由于经过蚂蚁多而遗留信息素浓度高,形成信息正向反馈。为了更好地发挥信息素引导作用,给出信息素局部更新和全局更新两种策略。

信息素局部更新策略中,对蚂蚁刚经过网格上的信息素以一定速率挥发,目的是让蚂蚁稍后尽量不选择刚经过的网格。因为大量蚂蚁会在路径起点和终点附近网格经过,若信息素不能及时挥发,蚂蚁则更倾向于在路径端点附近探索,降低寻优能力。局部更新策略如式(5)所示:

$$cell(r, c, l).pheromone = gama \times cell(r, c, l).pheromone \quad (5)$$

其中, $gama$ 为局部信息素残留因子,取值在 0~1 之间,取值越大信息素残留越多,已有路径对后续蚂蚁吸引力越强,实验表明, $gama$ 倾向于取较大值,能起到一定离散作用即可,取值在 0.8~0.9 较为合适。 $cell(r, c, l)$ 为蚂蚁刚经过的网格,其位置坐标为 (r, c, l) , $pheromone$ 为网格中的信息素。

信息素全局更新策略中,每只蚂蚁完成从起点到终点的寻路后,都会在所经过的路径网格上留下属于它的信息素浓度。先前研究对全部蚂蚁经过的路径做信息素更新,各蚂蚁遗留信息素浓度取决于路径长度^[8, 10]。该方法计算量较大,而且由于每次迭代产生的蚂蚁路径变化较大,导致信息素在网格空间中遗留过于分散,不利于搜索收敛。本文在每次迭代后仅对最优路径集合中的路径网格做信息素更新,这样不但计算量小,还使信息素遗留到更有希望是优化路径的网格中,全局更新策略如式(6):

$$cell(r, c, l).deta = \sum_{\substack{cell(r, c, l) \in path \\ path \in bestList}} Q / Length_{path} \quad (6a)$$

$$cell(r, c, l).pheromone = rou \times cell(r, c, l).pheromone + cell(r, c, l).deta \quad (6b)$$

其中, Q 为信息素常量, $bestList$ 为当前最优解集合, $Length_{path}$ 为路径长度, $deta$ 为 $bestList$ 解集中各蚂蚁路径在网格 (r, c, l) 上产生的信息素增量, rou 为全局信息素残留因子,取值在 0~1 之间,取值较小时,路径残留信息素少,搜索中对以往信息利用较少,取值较大时,路径残留信息素多,但搜索易陷入局部最优,为平衡算法性能, rou 取折衷值 0.5~0.7 较好。

(3) 方向选择

蚂蚁寻路的移动方向要综合启发式信息和信息素浓度确定,转移概率计算如式(7):

$$cell(r, c, l).pro[k] = \begin{cases} \alpha \times cell(r', c', l').pheromone + \\ \beta \times cell(r', c', l').heuristic, \text{邻居网格可达时} \\ 0, \text{邻居网格不可达时} \end{cases} \quad (7)$$

其中, $cell[r, c, l].pro[k]$, $k \in [0, 1, 2, 3, 4, 5]$ 表示蚂蚁沿某一方方向 k 移动的概率值。 $cell(r', c', l')$ 表示 $cell(r, c, l)$ 上下左右前后 6 个方向之一的邻居网格, $|r' + c' + l' - |r + c + l|| = 1$, 即当其中一个坐标变化 1 个距离位置,另两个坐标不变时,得到一个相邻网格。邻居网格不可达,要么邻居网格是障碍网格,要么是蚂蚁已探索过的网格。 α 和 β 的大小决定了方向转移概率对下一结点中信息素和启发式信息的相对重视程度,其不同大小的组合会影响算法的探索 and 开发能力,一般情况下,推荐 α 和 β 取相同值即可,例如 1。

为了避免蚁群寻路早熟收敛,还需引入随机转移机制,即蚂蚁以小概率随机选择方向,而以大概率按启发式信息和信息素浓度确定的最大概率方向选择方向,如式(8)所示:

$$Dir = \begin{cases} \max(cell(r, c, l).pro[k]), q \leq q_0 \\ roulette(\frac{cell(r, c, l).pro[k]}{\sum_{k \in [0, 1, 2, 3, 4, 5]} cell(r, c, l).pro[k]}), q > q_0 \end{cases} \quad (8)$$

其中, Dir 为确定的行进方向, $\max()$ 为最大值计算函数, $roulette()$ 是轮盘赌策略函数^[20]。 q_0 是阈值常数,若取值较小,则寻路算法随机性增强,不利于搜索过程收敛,若取值过大,则寻路算法缺乏随机性,不利于探索新的解空间,因此建议 q_0 取值为 0.9~0.97。

(4) 辅助点策略

理论上蚂蚁按式(3)~(8)可以找到起点 s 到终点 t 之间的路径,但在搜索开始时,布局空间中并未形成连续的 s 到 t 的信息素痕迹,蚂蚁更依赖启发式信息寻路,如果 s 和 t 位于巨大障碍两端,如前文图 1 所示,光靠距离引导的启发信息无法绕过障碍,即便加入折弯数和能量信息后,其绕障能力得到提高,但仍不够强大。为此可为蚁群寻路提出一种辅助点策略,即每只蚂蚁在寻找 s 到 t 的路径时需让其经过一个随机生成的辅助连接点 m (可限制 m 位于低能量区且不在障碍中),此时蚂蚁从 s 到 t 的寻路问题转化为 s 到 m 与 m 到 t 的寻路过程相加。若存在 s 到 m , 或 m 到 t 无法找到路径的情况,则重新生成 m 位置后再尝试,效果如图 3。默认情

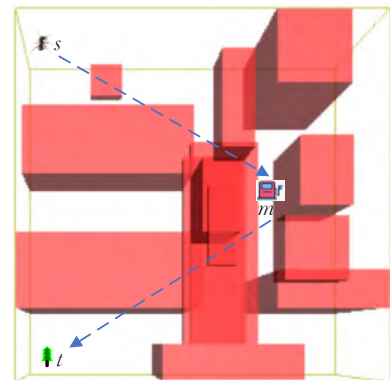


图3 辅助点寻路引导作用

Fig.3 Routing guidance of auxiliary point

况不需要辅助连接点,只有当蚂蚁无法在规定时间内生成路径时引入辅助连接点,辅助点的加入增加了寻路时间,但可以提高蚂蚁搜索能力。

2.1.2 优解保持策略

单目标优化算法基于目标函数的适应值来评价个体质量,通过观察发现适应值相同的蚂蚁路径布局方式不唯一,可将这些优解都保存,为设计师提供更多参考。将搜索到的最优解个体集保存到 *bestAnts* (列表)中,要求列表中个体适应值相同但布局路径不同(布局相同个体仅需保留一个),如果后续搜索发现适应值更好个体,则用新个体更新 *bestAnts* 列表,适应值相同但布局效果不同的个体会全部保留其中,搜索结束时 *bestAnts* 中存放的个体集即为最优个体集合, *bestAnts* 中个体能直接体现目标权重设置所反映的用户偏好。在不同次的算法运行中,用户通过调整各目标权重值,便可得到更多具有不同布局特征的优解集合。但为单目标优化方法适应度函数选取合适的目标权重系数需要一定的用户经验和试错成本,权重系数大小体现了各目标在路径评价中的重要程度,具有一定主观性,用户要在实验中将算法布局结果与人的布局期待结果进行比较,然后按期待调整各权重系数的比重关系,具体调整策略见 1.3 节式(3)后的文字说明。相比多目标优化算法不能事先设定管路布局偏好,只能在一次运行后被动获得 Pareto 解集,再在 Pareto 解集中选择符合偏好的布局方案,本文方法更符合用户实际使用体验。此外,如 1.3 节所述各目标“界限值”会更新的原因, *bestAnts* 中解的适应值要在每次迭代中重新计算一次,以便和当前迭代发现的解在相同标准下比较。

2.1.3 并行计算策略

蚁群算法具有内在并行性,即同一时刻蚁群个体均在执行寻路过程,因此算法设计和实现中可基于并行计算策略完成,在不降低寻路质量的同时提升搜索速度。本文基于 OpenMP 技术在多线程 CPU 上实现并行计算。

算法最耗时的往往就是循环代码,若循环中的处理过程也耗时,那么这个循环就要考虑并行优化。分析算法得到可并行执行的过程包括:蚂蚁个体生成和重置过程(GenerateAnts、ResetAnts)、蚂蚁寻路过程(MoveAnts)、蚂蚁路径局部爬山过程(ClimbHill4Ants)、蚂蚁路径评价过程(CalculateFitness、CalculateFitness4BestAnts)、网格环境信息素更新过程(UpdatePheromones)。这些过程的循环体中独立处理单个蚂蚁个体,各次循环间没有相关性。对于这种循环代码,使用 OpenMP 的 `#pragma omp parallel for` 指令,可将其后的 for 循环代码自动线程化,并将计算任务分配到不同 CPU 线程处理,方便在程序逻辑不变情况下加速计算。

需要说明的是,在蚂蚁寻路过程(MoveAnts)中会对信息素做局部更新,各蚂蚁对环境的影响是同时进行

的,也是相互独立的,这符合现实中蚂蚁种群的寻路行为,因此无需对蚂蚁更新网格信息素的代码做互斥处理。蚂蚁在寻路中最好不重复经过它曾到达过的网格,否则会使路径含有回路而降低效率,因此为每个网格 *cell* 设置一个状态数组 *pathStatus[N]*, *N* 为指定的 CPU 逻辑处理器数目,各逻辑线程上寻路的蚂蚁只更新它对应的状态 *cell.pathStatus[Id]* ($0 \leq Id \leq N$), *cell.pathStatus[Id]* 初始为 -1 表示 *cell* 不在该蚂蚁的路径上,为 0 表示 *cell* 已在该蚂蚁的路径上。MoveAnts 过程对 *pathStatus* 状态按线程不同区别处理,符合各蚂蚁路径相互独立的实际,MoveAnts 的并行化改进可有效提高寻路效率,甚至由于信息素并行更新机制更符合实际也会增进路径寻优效果。同时,即便对单核 CPU,算法逻辑无需修改也可正常执行。

OpenMP 的实际加速效果与编译器的优化配置有关系^[21]。当计算核数增加到一定数目后不会持续缩短时间,反而可能增加时间,这是由于线程通信也需要成本,且参与计算内核过多会导致 CPU 升温,部分 CPU 会做降频保护。

2.2 算法流程

改进蚁群优化算法 MdACO 的整体流程如图 4。其中,GenerateAnts 用于创建路径为空的初始蚂蚁对象,ResetAnts 用于在每次迭代前将各蚂蚁路径重置为空,MoveAnts 完成各蚂蚁从起点 *s* 到终点 *t* 的寻路过程(若启用辅助点策略,则分两段完成寻路),ClimbHill4Ants 用于对已得到的蚂蚁路径做局部爬山,CaculateFitness 和 CalculateFitness4BestAnts 用于计算蚂蚁种群路径的适应值,UpdateBestAnts 和 RemoveSameAnts 完成最优解的迭代更新(见本文 2.1.2 小节描述),UpdatePheromones 用于更新环境信息素分布。

ResetAnts 和 RemoveSameAnts 方法逻辑较为简单,此处仅对 MdACO 关键环节进行重点说明。

MoveAnts 为算法的核心方法,各蚂蚁根据启发式信息和信息素信息完成寻路,一只蚂蚁在两点间的寻路过程如图 5,若启用辅助连接点,则以下寻路流程执行两次后再拼接路径 ($path_{st} \leftarrow path_{sm} + path_{mt}$)。

上述流程中判定路径 *path* 太差的依据是蚂蚁路径长度大于空间长宽高之和的 2 倍,这种情况即便接受该路径,由于路径质量不高,也会随着进化被淘汰,不如重新生成更好的路径参与竞争。

其他方法以伪代码形式描述其逻辑如下。

(1) GenerateAnts 方法

```
输入: startPos, endPos //起止位置,类型为(r, c, l)坐标
输出: vector<Ant> ants //当前蚂蚁种群集合
{
    #pragma omp parallel for //for循环并行化指令
    for (i=0; i<ants.size(); i++)
```

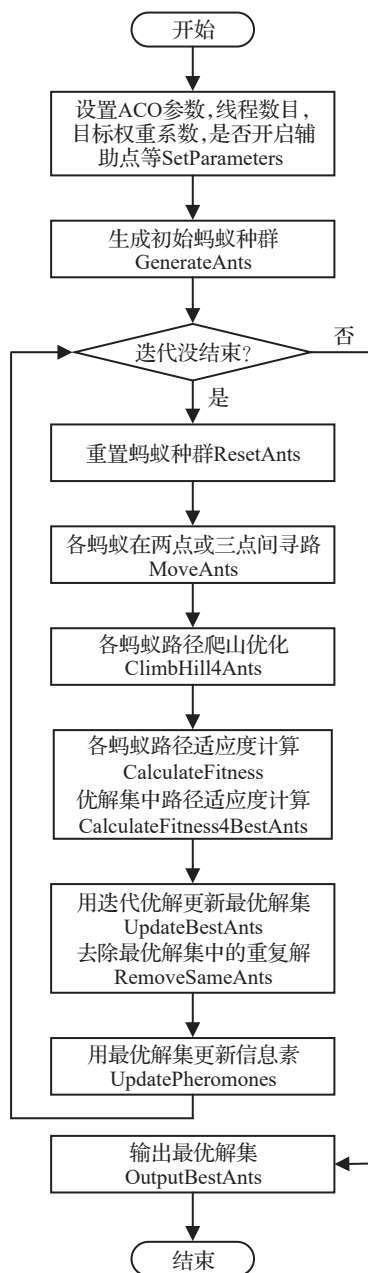


图4 MdACO算法流程

Fig.4 Process of MdACO algorithm

```

{
    step1: 创建蚂蚁 ant, 设置连接点数(0或1);
    step2: 生成此 ant 连接点位置;
    step3: 将 ant 加入 ants 集合;
}
step4: 返回 ants;
}

(2)ClimbHill4Ants 方法
输入: vector<Ant> ants, int num //num 爬山次数
输出: vector<Ant> ants
{
    #pragma omp parallel for
    for (i=0; i<ants.size(); i++)
    {

```

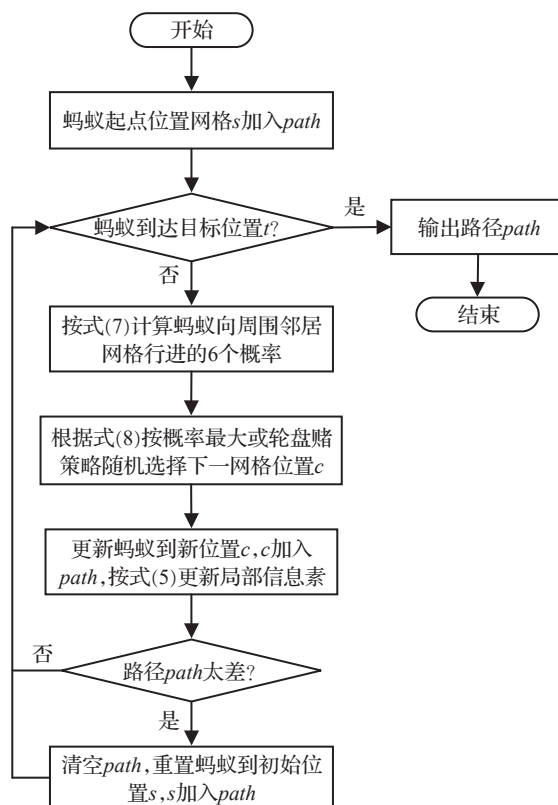


图5 蚂蚁寻路流程

Fig.5 Path-finding process of ant

```

step1: ant ← ants[i];

```

step2: 尝试对 ant 做 num 次爬山优化 (按文献[15]中 2.2.3 小节方法随机修改部分路径折弯间的连接方式), 若新路径适应值增加, 则 ant 采用新路径, 否则不变;

```

}
step3: 返回更新后的 ants;
}

```

(3) CaculateFitness 方法

输入: vector<Ant> ants

输出: vector<Ant> ants

```

{
    #pragma omp parallel for
    for (i=0; i<ants.size(); i++)
    {
        step1: 计算蚂蚁 ant[i] 各子目标值;
        step2: 尝试用 ant[i] 中子目标值更新各子目标全局最大值 objMaxValueSet 和最小值 objMinValueSet;
    }
    #pragma omp parallel for
    for (i=0; i<ants.size(); i++)
    {
        step3: 将 ant[i], objMaxValueSet, objMinValueSet 带入前文式 3, 计算 ant[i] 的适应值;
    }
    step4: 返回 ants;
}

```


CalculateFitness4BestAnts 与 CaculateFitness 方法逻辑基本相同,区别在于前者对最优解集合 bestAnts 中蚂蚁计算适应度,后者对种群全部蚂蚁 ants 计算适应度。

(4)UpdateBestAnts 方法

输入:vector<Ant> ants, //当前蚂蚁种群集合

vector<Ant> bestAnts //当前最优蚂蚁集合

输出:vector<Ant> bestAnts

{

step1:找出 ants 中最好的蚂蚁 it_bestAnt(适应值最大,路径最好);

if (it_bestAnt.fitness>bestAnts[0].fitness)

{

step2:清空 bestAnts,即 bestAnts.clear();

step3:将 it_bestAnt 放入 bestAnts 集合;

}

else if (it_bestAnt.fitness==bestAnts[0].fitness)

{

step4:将 it_bestAnt 放入 bestAnts 集合;

}

else { //无需更新 bestAnts }

Step5:返回更新后的 bestAnts;

}

(5)UpdatePheromones 方法

输入:vector<Ant> bestAnts //当前最优蚂蚁集合

输出:Cell[R][C][L] spaceCells //网格空间数组

{

#pragma omp parallel for

for (i=0;i<bestAnts.size();i++)

{

step1:对蚂蚁 bestAnts[i]路径上所有网格用式(6a)

计算信息素增量 cell(r,c,l).deta,(r,c,l)代表网格坐标;

}

#pragma omp parallel for

for (cell(r,c,l) in spaceCells) //遍历空间中各网格

{

step2:对网格 cell(r,c,l)用式(6b)计算更新后的信

息素 cell(r,c,l).pheromone;

}

step3:返回更新信息素后的网格空间 spaceCells;

}

3 仿真实验

为了分析算法表现,通过具体算例验证算法有效性和先进性。

3.1 算例及环境设置

为与同类算法对比,基于前期研究^[15-17,22]的算例布局管路。布局空间为边长100的立方体,内部有13个矩形障碍物,设空间中心点坐标为(0,0,0),障碍物各对角顶点坐标对如表1所示。空间划分为100×100×100的网格,空间左下后角处网格位置记为(0,0,0),靠近空间边

界和障碍物表面的区域为低能区,最靠近支撑表面的网格能量为0,每远离一层网格能量加5,5层网格之外的网格能量都是25。选择距离较远且被大物体阻隔的两个管路接口布局管路,管路P的接口网格坐标为(99,0,99)~(99,46,0)。

表1 障碍位置

Table 1 Obstacle location

序号	障碍物对角顶点坐标
1	(-45,-50,20)~(-30,20,0)
2	(0,-50,30)~(50,10,10)
3	(0,-50,-10)~(50,50,-30)
4	(-40,-50,50)~(-20,50,30)
5	(-50,-50,-20)~(-20,50,-30)
6	(-20,-10,-40)~(0,0,20)
7	(-50,-50,-20)~(-30,-20,0)
8	(-30,-50,-50)~(10,30,-40)
9	(-20,-50,-40)~(0,-40,20)
10	(-10,0,-10)~(0,50,10)
11	(30,-50,50)~(20,-40,40)
12	(-10,-50,50)~(-20,-10,20)
13	(-14,-10,-16)~(-6,-16,8)

运行环境:MS VC++ 2019,Intel® Core™ i5-8265U CPU@1.60 GHz(逻辑处理器数为8),RAM 24 GB,OS Win10 x64。

参数设置:蚂蚁数 $antCount = 50$,进化代数 $itCount = 500$,信息素常数 $Q = 100$,信息素信息权重 $\alpha = 1$,启发式信息权重 $\beta = 1$,方向转移概率阈值 $q_0 = 0.95$,全局信息素残留因子 $rou = 0.6$,局部信息素残留因子 $gama = 0.8$ 。MdACO的参数推荐设置范围和灵敏性分析与前期研究MOACO算法相似^[16]。

3.2 实验结果及分析

3.2.1 并行计算效果验证

为观察OpenMP并行计算对MdACO算法的加速效果,设置编译器OpenMP支持为开启,评价函数权重为 $w_{length} = 0.4$, $w_{bend} = 0.4$, $w_{power} = 0.2$, $w_{pocket} = 500$, $w_{times} = 500$,观察编译器在开启速度优化(/O2)和禁用优化(/Od)时,线程数目逐渐增加情况下算法表现。统计5次计算的均值如表2。

表2 OpenMP加速效果

Table 2 Acceleration effects using OpenMP

线程数	编译优化设置	平均获得解数	耗时/s
1	O2/Od	5.4/5.4	73.6/195.5
2	O2/Od	6.0/6.0	53.2/126.2
3	O2/Od	6.0/7.4	46.4/109.1
4	O2/Od	6.0/11.6	46.2/91.9
5	O2/Od	6.6/10.8	45.2/87.9
6	O2/Od	8.4/12.0	41.6/82.4
7	O2/Od	8.4/12.0	44.3/77.0
8	O2/Od	8.0/8.4	43.2/95.8

分析表2中数据可知,开启OpenMP后,当参与计算线程数目从1到8增加时,开启或禁用编译器优化,计算时间基本都是先减少后略增加,在O2(速度优化)配置下时,最少时间出现在6线程,其次是7线程,在Od(禁用优化)配置下,最少时间发生在7线程,其次是6线程,而在获得平均优解个数方面,最好情况都是在6线程或7线程,且此时Od下获得优解个数明显多于O2下个数。从以上结果看出,开启编译器优化在提升计算效率的同时也会影响算法寻优表现(优解个数),但不会影响算法正确性,这与编译器优化技术有关^[21]。此外,计算耗时开始时随着线程增加而减少,而后期略增加,这是因为OpenMP将for循环的计算任务自动分配到不同线程,当全部线程结束后才继续向下执行,线程间的同步操作也会占用时间,因此在接近CPU全部算力时耗时反而略增加。

以上算例中发现的最优路径指标数据都相同,即长度为246,折弯数为6,能量值为0,并满足其他目标和约束,但管路布局效果不同,算法得到的全部12种布局如图6,其中包括多优解同时显示效果(前2个子图)和各优解单独显示效果(后12个子图)。

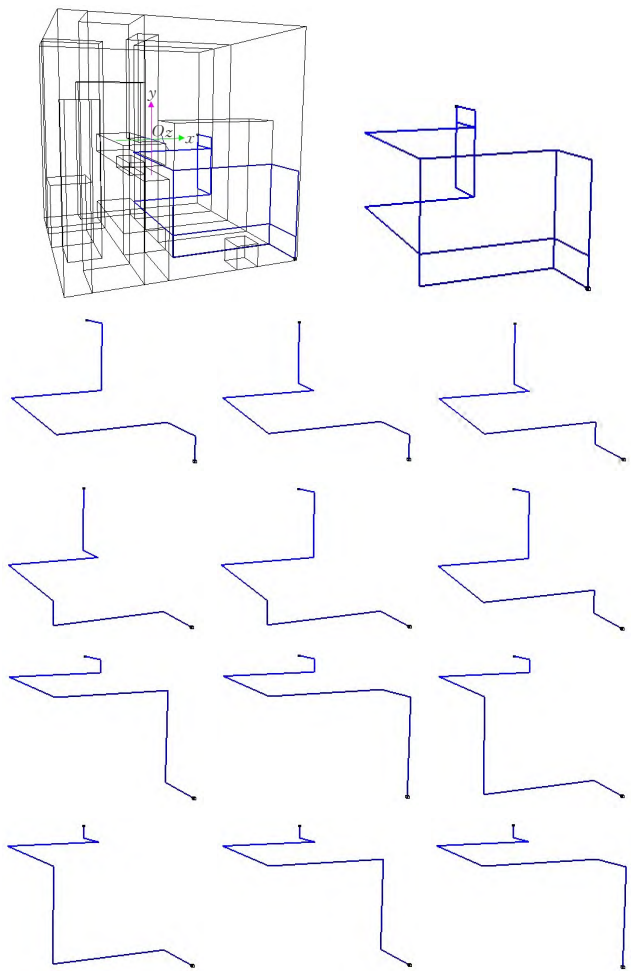


图6 优解布局效果

Fig.6 Layout effects of optimal solutions

按上述实验结论,为了避免编译器优化对算法产生影响并尽可能提高计算速度,后续实验中均禁用编译器优化(/Od),开启OpenMP支持,设置7线程参与计算。在用于实际设计的软件产品中,若追求计算速度可开启编译优化(/O2),寻优性能会略有降低。

3.2.2 辅助点效果验证

为观察MdACO中辅助连接点作用,在同样权重设置 $w_{length} = 0.4, w_{bend} = 0.4, w_{power} = 0.2, w_{pocket} = 500, w_{times} = 500$,禁用编译器优化(/Od),7线程参与计算下,每种启发式配置运行5次,统计结果如表3。

表3 辅助点效果测试

Table 3 Effect test for using auxiliary point

启发式函数	辅助点	平均优解数	平均耗时/s	平均收敛代数
<i>dist + bends + power</i>	有	12.0	114.1	99.0
<i>dist + bends + power</i>	无	12.0	75.2	238.6
<i>dist + bends</i>	有	12.0	95.6	187.8
<i>dist + bends</i>	无	10.8	66.5	336.8
<i>dist</i>	有	6.0	472.9	177.8
<i>dist</i>	无	0	∞	∞

分析表3数据可知,引入辅助连接点后,算法发现优解的能力将增强,尤其在启发式信息减少后,当仅用距离*dist*引导蚂蚁搜索,没有辅助点时蚂蚁将无法绕过障碍形成路径,而有辅助点时算法仍能发现一半数目优解,但耗时会急剧增多。正如2.1.1节分析,引入辅助点将增加一次寻路过程,所以平均耗时增加较多,但好处是平均收敛速度更快,综合而言,在相同启发式函数配置下,有辅助点可以更早发现更多优解。

3.2.3 信息素更新机制验证

信息素分布会影响蚂蚁寻路方向选择,进而影响算法寻路能力,为验证所提信息素更新机制有效性,做如下对比实验。三组实验中仅信息素更新机制不同,目标权重、编译器设置和计算线程数量与3.2.2节相同,引入一个辅助点,其他参数同前文。PUM-1组对蚂蚁寻路刚经过的网格做信息素挥发(2.1.1局部信息素更新),并在迭代中仅对优解路径集的网格做信息素更新(2.1.1全局信息素更新);PUM-2组不加入局部信息素更新,仅使用全局信息素更新;PUM-3组保留局部信息素更新,但采用文献[8]提出的全局信息素更新机制,迭代中对所有个体路径网格做信息素更新,但根据路径质量选择不同的更新权重系数*c*,即*c*=2(迭代最优解或当前全局最优解)或0(迭代最差解)或1(其他解)。每种配置运行5次,实验结果如表4。

分析表4可知,无论算法获得优解质量、优解个数、最优解比例,还是算法收敛速度和执行速度方面,本文PUM-1机制都是最高效的;PUM-2机制由于没有局部信息素更新机制,寻优能力有所下降,算法耗时和收敛代数相对PUM-1均有增加,且获得优解个数减少,但获

表4 信息素更新机制效果对比

Table 4 Effect comparison between different pheromone update mechanisms

信息素更新机制	平均长度	平均折弯数	平均能量值	平均收敛代数	平均耗时/s	获得优解数 (5次合计)	含最优解数 (5次合计)	获得最优解概率/%
PUM-1	246.0	6.0	0	133	120.7	60	60	100
PUM-2	246.0	6.0	0	312	162.9	46	46	100
PUM-3	246.2	6.2	0	307	571.8	8	6	75

得最优解比例仍然较高;PUM-3 机制采用文献[8]中策略对迭代中全部个体路径进行信息素更新,使信息素分布相对分散,也降低了启发式信息在寻路中的引导作用,导致算法在发现优解能力(个数和质量)和耗时等各方面均出现较大下降。

3.2.4 与其他布局算法对比验证

将MdACO算法(在不同权重下、有辅助点、OpenMP支持,7线程计算)与两种多目标算法 MOACO^[16]和 NSGA-II^[15](多目标算法参数设置与文献中一致,此实验中禁用编译优化/Od)分别运行5次。统计获得的平均优解数、计算耗时和收敛代数,结果如表5,当MdACO的G3~G5组权重系数 w_{bend} 或 w_{power} 取0时,式(3)也不加入与该权重对应的启发信息 $bends(c_{t-1}, c_t, c_{t+1})$ 或 $power(c_{t+1})$,G6组权重系数 w_{length} 和 w_{power} 取0时,式(3)中仅不加入 $power(c_{t+1})$,因为若没有 $distance(c_{t+1}, c_{\text{end}})$ 引导搜索,则无法找到可行路径。MOACO和NSGA-II是多目标算法,不需要配置目标权重系数,因此系数列标记为NA。为不同组别实验各选择一个布局效果,如图7所示。其中图7(a)为MdACO-G1/G2组的效果图,含12个解,路径长246,折弯数6,能量值0;图7(b)为MdACO-G3组的一个布局效果,含52个解,路径长都为

246,折弯数都为5,能量值都大于0;图7(c)为MdACO-G4组的一个布局效果,含95个解,路径长都为246,平均折弯数大于6,能量值都为0;图7(d)为MdACO-G5组的一个布局效果,含62个解,路径长都为246,平均折弯数和平均能量值较高;图7(e)为MdACO-G6组的一个布局效果,含15个解,路径长都大于246,折弯数最少,都为4,平均能量值大于0;图7(f)为MOACO的一个布局效果,含23个解,图7(g)为NSGA-II的一个布局效果,含15个解,多目标算法的解个体之间互不占优,用户可根据需要选择。

表5 多目标算法结果对比

Table 5 Result comparison between multi objective algorithms

算法	$w_{\text{length}}, w_{\text{bend}}, w_{\text{power}}, w_{\text{pocket}}, w_{\text{times}}$		平均优	平均耗	平均收
			解数	时/s	敛代数
MdACO	G1	0.6,0.3,0.1,500,500	12.0	116.7	148.0
	G2	0.1,0.3,0.6,500,500	12.0	116.9	110.6
	G3	0.5,0.5,0.0,500,500	35.6	111.1	428.6
	G4	0.5,0.0,0.5,500,500	76.0	528.0	492.2
	G5	1.0,0.0,0.0,500,500	73.0	409.6	495.6
	G6	0.0,1.0,0.0,500,500	12.8	120.1	435.2
MOACO		NA	19.8	146.7	177.2
NSGA-II		NA	15.0	394.5	12.6

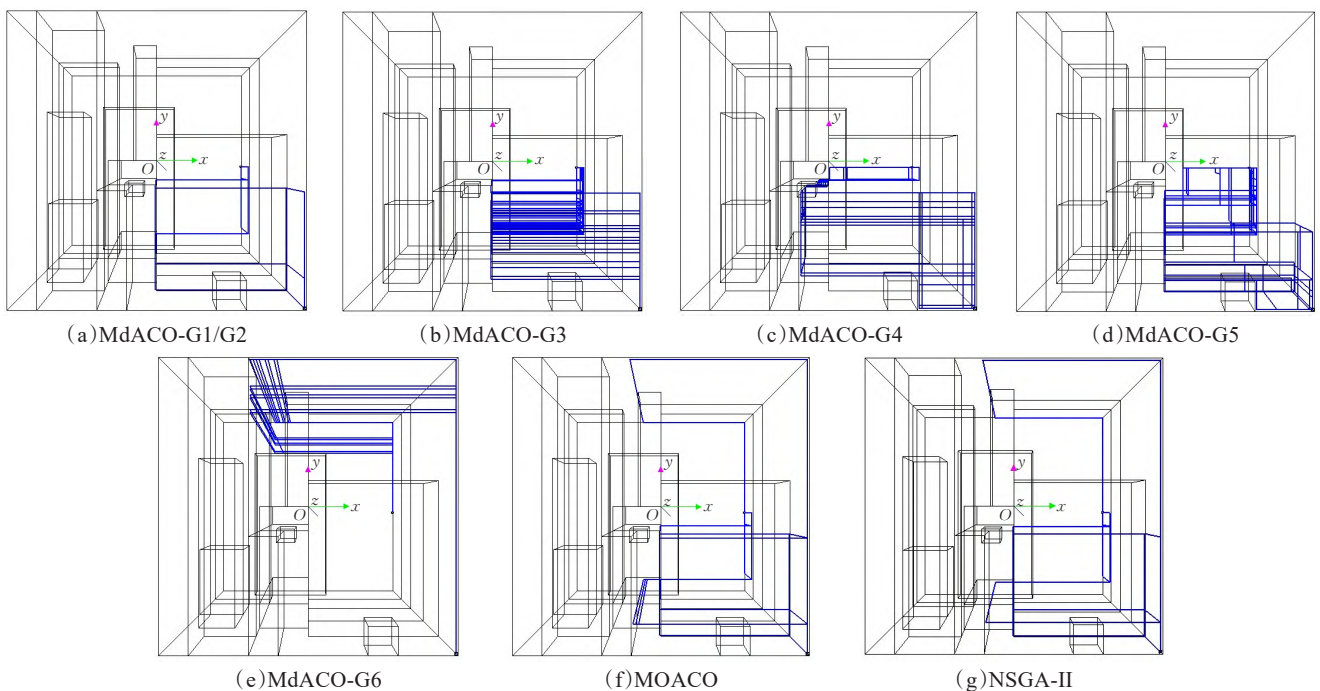


图7 MdACO、MOACO、NSGA-II 布局效果

Fig.7 Layout effects of MdACO, MOACO, NSGA-II

表6 单目标算法结果对比
Table 6 Result comparison between single objective algorithms

算法	平均长度	平均折弯数	平均能量值	平均收敛代数	运行内存/MB	平均耗时/s	平均获得解数	解中最优解概率/%
MdACO(G1)	246.0	6.0	0	148	649	116.7	12	100
GA-Dong	246.0	6.7	0	368	376	4.4	1	30
GA-Sui	246.0	7.1	4	77	406	629.7	1	30
PSO-Dong	256.2	9.5	176	594	374	3.1	1	0

分析表5中数据可得出以下结论:

(1)为MdACO配置不同目标权重系数会得到布局特点不同的多种优化解。当各权重系数取值不为0但相对大小变化时(G1~G2),算法表现稳定,均能在较短时间,用较少收敛代数得到目标均衡的12个优解;若将折弯权重或能量权重设为0(G3~G6),并减少相应启发式信息,则算法收敛的平均代数将增加(>400),但由于约束减少,发现的解会变多(>12);算法执行时间与启发式函数有关,若启发信息中没有折弯信息,则计算耗时最多(>400 s),否则均在120 s左右。

(2)多目标算法MOACO和NSGA-II计算中不需要为各优化目标配置权重系数,算法根据不同解间的支配关系确定优劣^[15-16],MOACO相比NSGA-II寻优能力更强(19.8>15)、耗时更少,但NSGA-II收敛速度更快。本文的MdACO算法相比以上多目标算法,可根据权重设置不同产生符合用户偏好的优解,能更好满足实际工程中为不同布管场景选择不同设计方案的需求。相比多目标算法,MdACO在不同权重下多次运行可给出的布管方案更多。

此外,将本文MdACO算法与基于遗传算法(GA-Dong^[22]、GA-Sui^[23])和粒子群优化算法(PSO-Dong^[24])的其他三种单目标布管算法进行对比(算法参数设置和更多计算结果详见文献[22]),各算法在相同布局空间中对相同管路 $P(99,0,99) \sim (99,46,0)$ (文献[22]中管路B2)进行布局,统计结果如表6所示,其中MdACO算法采用表5中G1组权重设置。分析数据可知,实验中本文MdACO算法每次运行均以100%概率找到12个不同布局的最优解(效果见图6和图7(a)),而其他单目标算法每次只给出一种优解且该优解是最优解的概率不如MdACO算法高;由于MdACO获得的解都是算法最优解,因此从管路布局路径的平均长度、平均折弯数目、平均能量值等优化指标看,也是本文算法更好;从平均收敛代数来看,MdACO算法仅不如GA-Sui算法,但好于另外两种算法;从运行时内存来看,MdACO算法占用内存约是其他算法的1.6~1.7倍,这与MdACO算法设置的种群规模(50)略大于其他单目标算法的种群规模(40)且算法运行中要保存优解集合有关;从平均耗时来看,MdACO算法效率好于同样基于变长编码策略的GA-Sui算法,但远不如基于定长编码策略的GA-Dong算法和PSO-Dong算法,这说明在对效率要求较高的情况下,定

长编码可极大降低计算量。通过对比实验说明MdACO算法在对运算时间要求不是很高的情况下,其求解的总体优化性能更有优势。

4 结语

提出的改进蚁群优化船舶管路布局算法(MdACO),通过对适应度函数做规范化处理,降低了权重系数在(0,1]区间变化时对算法性能的影响,使用户设置权重系数更容易;将折弯数和能量值引入启发式信息,使蚂蚁寻路性能和路径质量得到明显提高;基于局部和全局信息素改进更新策略,提升了信息素对路径搜索的引导作用;此外,辅助点策略提高了算法绕障能力和收敛效率,基于OpenMP的并行计算在不改变算法逻辑情况下发挥出CPU多核优势,提升了计算速度和寻优能力。实验结果表明,本文MdACO算法在优解发现、计算速度等方面与基于多目标的船舶布管算法(NSGA-II^[15]、MOACO^[16])相比具有一定优势,与基于遗传算法^[22-23]和粒子群优化^[24]的其他单目标算法相比,其整体寻优能力也更有优势,最重要的是能够利用优解保持策略在一次运行中保存适应值相同而布局效果不同的优解集合,在不同权重设置下多次运行算法生成符合用户偏好的布局方案更多。

未来工作计划:利用高性能显卡的CUDA技术对算法做深度并行化改进;探索基于机器学习策略的新型船舶管路布局方法。此外,为提高MdACO算法搜索性能,后期可采用文献[12]中的自适应转移概率阈值策略和动态信息素更新机制对算法做进一步优化。

参考文献:

[1] Product data representation and exchange-part: 217, application protocol: ship piping: ISO/CDC 10303-217[S]. New York: SCRA, 1996.

[2] PARK J H, STORCH R L. Pipe-routing algorithm development: case study of a ship engine room design[J]. Expert System with Applications, 2002, 23(3): 299-309.

[3] 柳强, 刘贝贝, 于嘉鹏, 等. 考虑双联卡箍约束的航空发动机多管束束敷优化方法[J]. 机械工程学报, 2021, 57(19): 218-229.

LIU Q, LIU B B, YU J P, et al. Multi-pipe bundle layout optimization for aero-engine considering double-clamp constraints [J]. Journal of Mechanical Engineering, 2021, 57(19): 218-229.

- [4] 苏锦成, 王振中, 贾小攀, 等. 基于A*算法的核电厂工艺管道自动布局方法[J]. 核科学与工程, 2022, 42(1): 129-135.
SU J C, WANG Z Z, JIA X P, et al. Automatic layout of pipelines for process systems of nuclear power plant based on A* algorithm[J]. Nuclear Science and Engineering, 2022, 42(1): 129-135.
- [5] 王发麟. 复杂机电产品线缆敷设若干关键技术研究[D]. 南京: 南京航空航天大学, 2018.
WANG F L. Research on key technologies of cable harness wiring for complex mechatronic products[D]. Nanjing: Nanjing University of Aeronautics and Astronautics, 2018.
- [6] 林韩熙, 向丹, 欧阳剑, 等. 移动机器人路径规划算法的研究综述[J]. 计算机工程与应用, 2021, 57(18): 38-48.
LIN H X, XIANG D, OUYANG J, et al. Review of path planning algorithms for mobile robots[J]. Computer Engineering and Applications, 2021, 57(18): 38-48.
- [7] ASMARA A. Pipe routing framework for detailed ship design [D]. Delft, The Netherlands: Delft University of Technology, 2013.
- [8] 范小宁. 船舶管路布局优化方法及应用研究[D]. 大连: 大连理工大学, 2006.
FAN X N. A study of optimization methods for ship pipe routing design and applications[D]. Dalian: Dalian University of Technology, 2006.
- [9] FAN X N, LIN Y, JI Z S. Ship pipe routing design using the ACO with iterative pheromone updating[J]. Journal of Ship Production and Design, 2007, 23(1): 36-45.
- [10] JIANG W Y, LIN Y, CHEN M, et al. A co-evolutionary improved multi-ant colony optimization for ship multiple and branch pipe route design[J]. Ocean Engineering, 2015, 102: 63-70.
- [11] WANG Y L, YU Y Y, LI K, et al. A human-computer cooperation improved ant colony optimization for ship pipe route design[J]. Ocean Engineering, 2018, 150: 12-20.
- [12] LIU C, WU L, HUANG X D, et al. Improved dynamic adaptive ant colony optimization algorithm to solve pipe routing design[J]. Knowledge-Based Systems, 2022, 237: 107846.
- [13] NIU W T, SUI H T, NIU Y X, et al. Ship pipe routing design using NSGA-II and coevolutionary algorithm[J]. Mathematical Problems in Engineering, 2016: 7912863.
- [14] SUI H T, NIU W T, NIU Y X, et al. Pipe-assembly approach for ships using modified NSGA-II algorithm[J]. Computer Aided Drafting, Design and Manufacturing, 2016, 26(2): 34-42.
- [15] 董宗然, 王法胜, 楼偶俊, 等. 基于改进NSGA-II的船舶管路路径设计[J]. 计算机集成制造系统, 2022, 28(4): 1129-1142.
DONG Z R, WANG F S, LOU O J, et al. Ship pipe route design based on improved NSGA-II[J]. Computer Integrated Manufacturing Systems, 2022, 28(4): 1129-1142.
- [16] DONG Z R, BIAN X Y, ZHAO S. Ship pipe route design using improved multi-objective ant colony optimization[J]. Ocean Engineering, 2022, 258: 111789.
- [17] DONG Z R, BIAN X Y. Ship pipe route design using improved A* algorithm and genetic algorithm[J]. IEEE Access, 2020, 8: 153273-153296.
- [18] DORIGO M, STÜTZLE T. Ant colony optimization[M]. Cambridge: MIT Press, 2004.
- [19] 宁伟康. 进化多目标优化算法研究及其应用[D]. 西安: 西安电子科技大学, 2018.
NING W K. Research on evolutionary multi-objective optimization algorithm and its application[D]. Xi'an: Xidian University, 2018.
- [20] GABRIELE D L. Roulette selection in genetic algorithms [EB/OL]. (2020-10-19) [2022-08-08]. <https://www.baeldung.com/cs/genetic-algorithms-roulette-selection>.
- [21] MATT G. Optimizations in C++ compilers[J]. Communications of the ACM, 2020, 63(2): 41-49.
- [22] 董宗然, 楼偶俊, 管官. 基于改进遗传算法的船舶管路布局设计[J]. 计算机工程与应用, 2020, 56(19): 252-260.
DONG Z R, LOU O J, GUAN G. Ship pipe route design based on improved genetic algorithm[J]. Computer Engineering and Applications, 2020, 56(19): 252-260.
- [23] 隋海腾. 基于改进遗传算法的船舶管路设计[D]. 天津: 天津大学, 2015.
SUI H T. Adaptive genetic algorithm based ship pipe design [D]. Tianjin: Tianjin University, 2015.
- [24] DONG Z R, LIN Y. A particle swarm optimization based approach for ship pipe route design[J]. International Shipbuilding Progress, 2017, 63(1/2): 59-84.